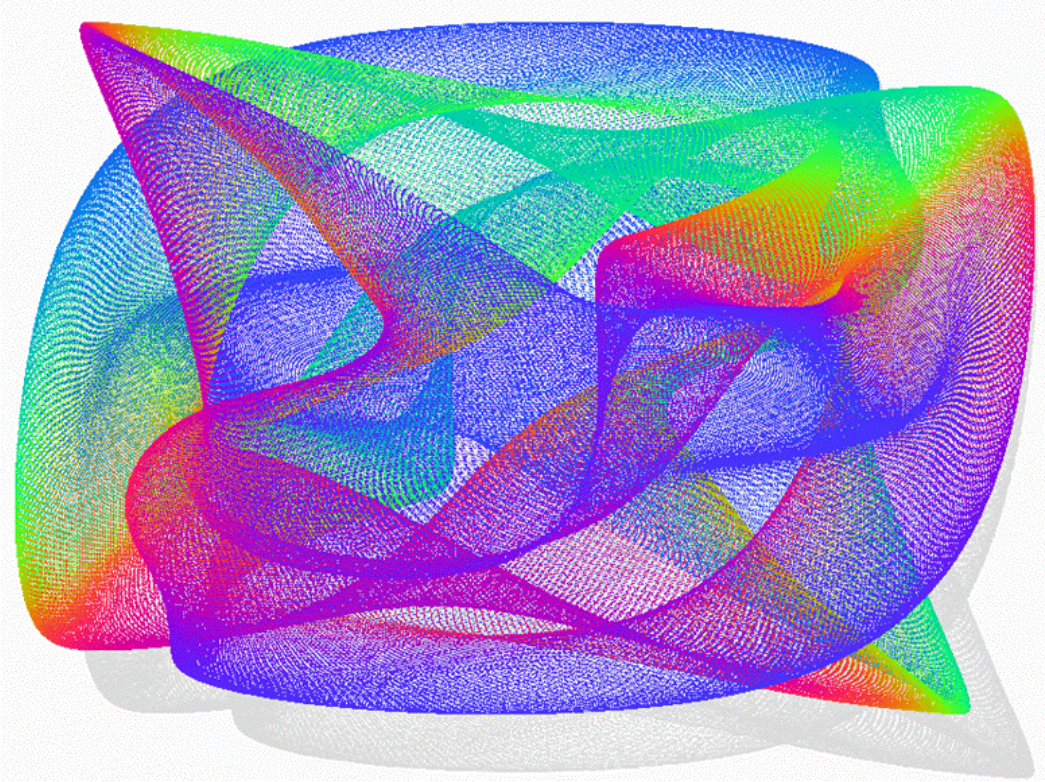
USC
UNIVERSIDADE DE SANTIAGO DE COMPOSTELA



Métodos de aprendizaje automático

Juan M. Ruso
Dpto. Física Aplicada
juanm.ruso@usc.es

Sprott, J. C. Artifical Neuronal Net Attractors. Chaos & Graphics, 22, 1998, 143

Inteligencia artificial (AI): disciplina dentro de las ciencias de la computación que trata de cómo las máquinas pueden imitar la inteligencia humana.

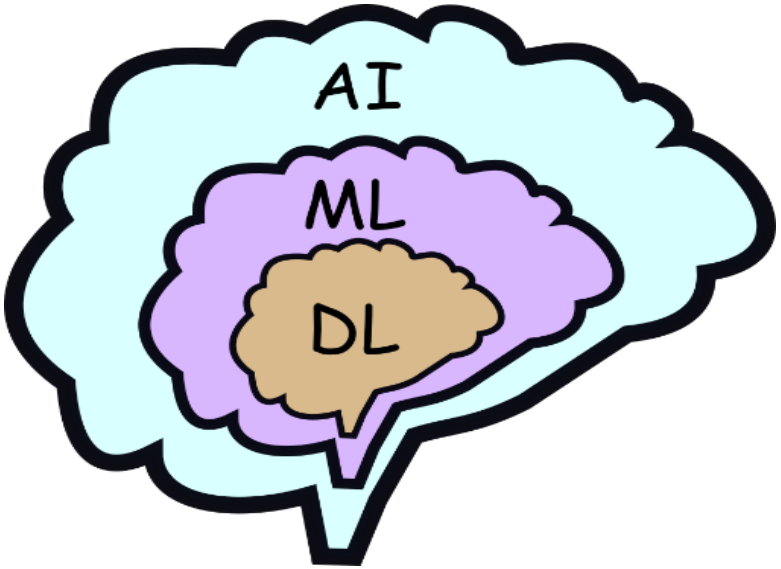
- **Métodos probabilísticos**: se usan cuando no estamos seguros, predecir si lloverá mañana con un porcentaje de probabilidad
- **Computación evolutiva**: imita la evolución natural, un programa prueba distintas soluciones y se quedaran las mejores, parecido a cómo sobreviven las especies más adaptadas
- **Sistemas difusos**: permiten trabajar con términos vagos, un aire acondicionado que no dice solo "frío" o "calor", sino "un poco caliente" o "bastante frío".
- **Aprendizaje automático (ML)**: las máquinas aprenden de datos, Netflix aprende qué series recomendarte según lo que has visto antes.

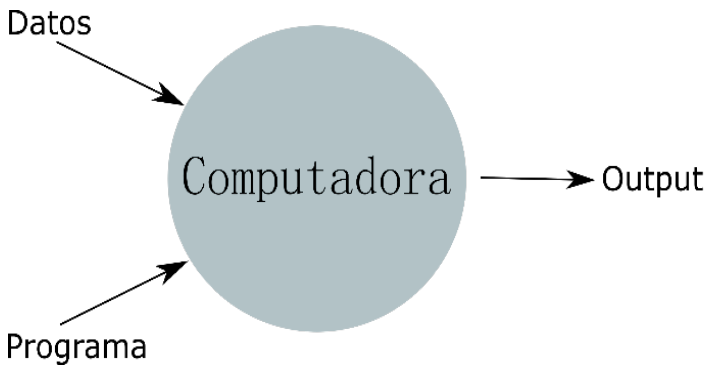
ML: colección de algoritmos y técnicas para diseñar sistemas que aprenden de los datos (predicciones o patrones).

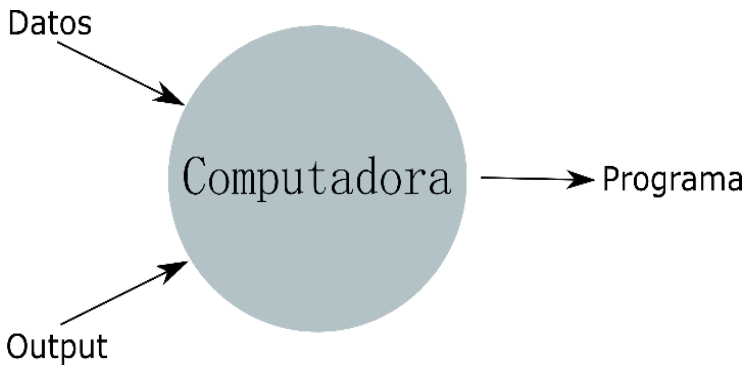
Datos: inciertos, inconsistentes, incompletos e imprecisos

50s: Arthur Samuel "Some Studies in Machine Learning Using the Game of Checkers".

Deep Learning: uso de ANN.

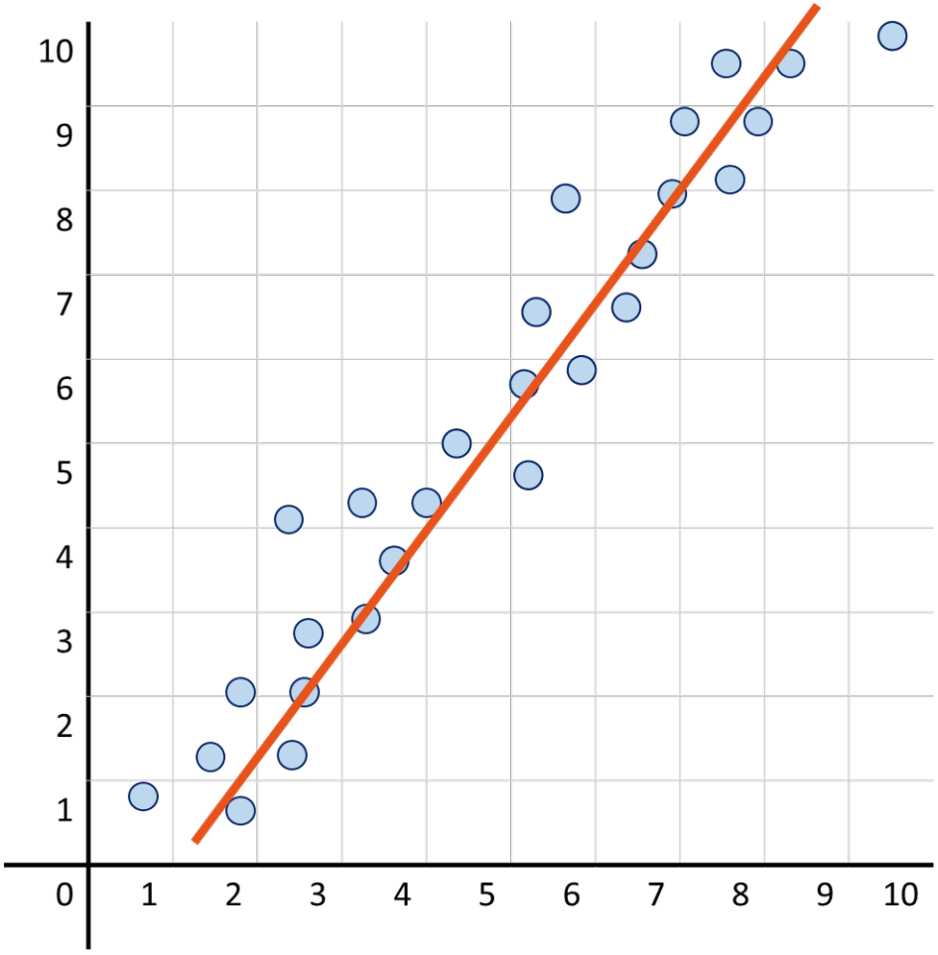






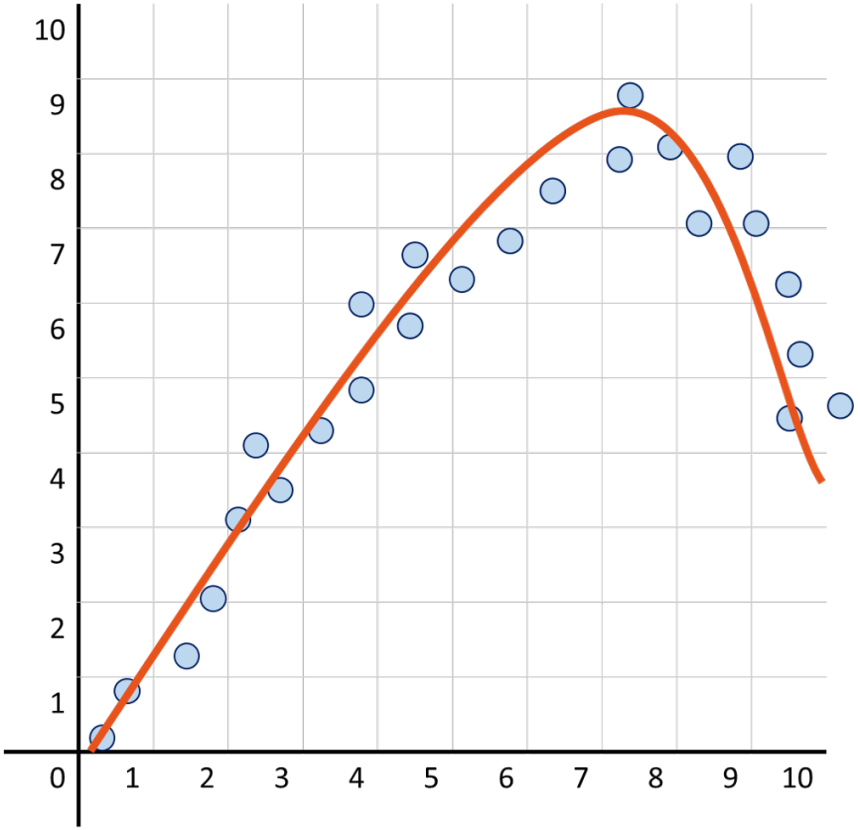
Regresión lineal

Método matemático que modela la relación entre una variable independiente "**x**" y una variable dependiente "**y**" por medio de una expresión lineal que busca asemejarse a en medida de lo posible a los datos reales.



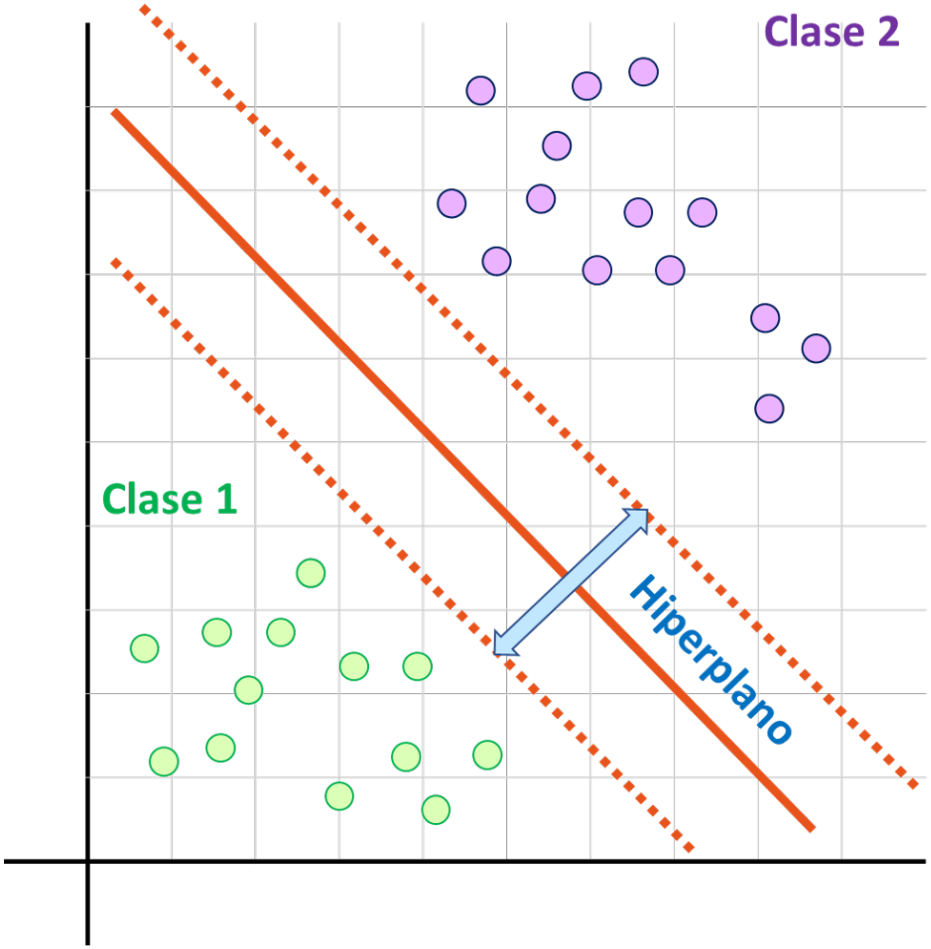
Regresión polinomial

La regresión polinomial es muy similar a la regresión lineal con la diferencia que el modelo matemático que se intenta construir es un "**polinomio de grado-n**".



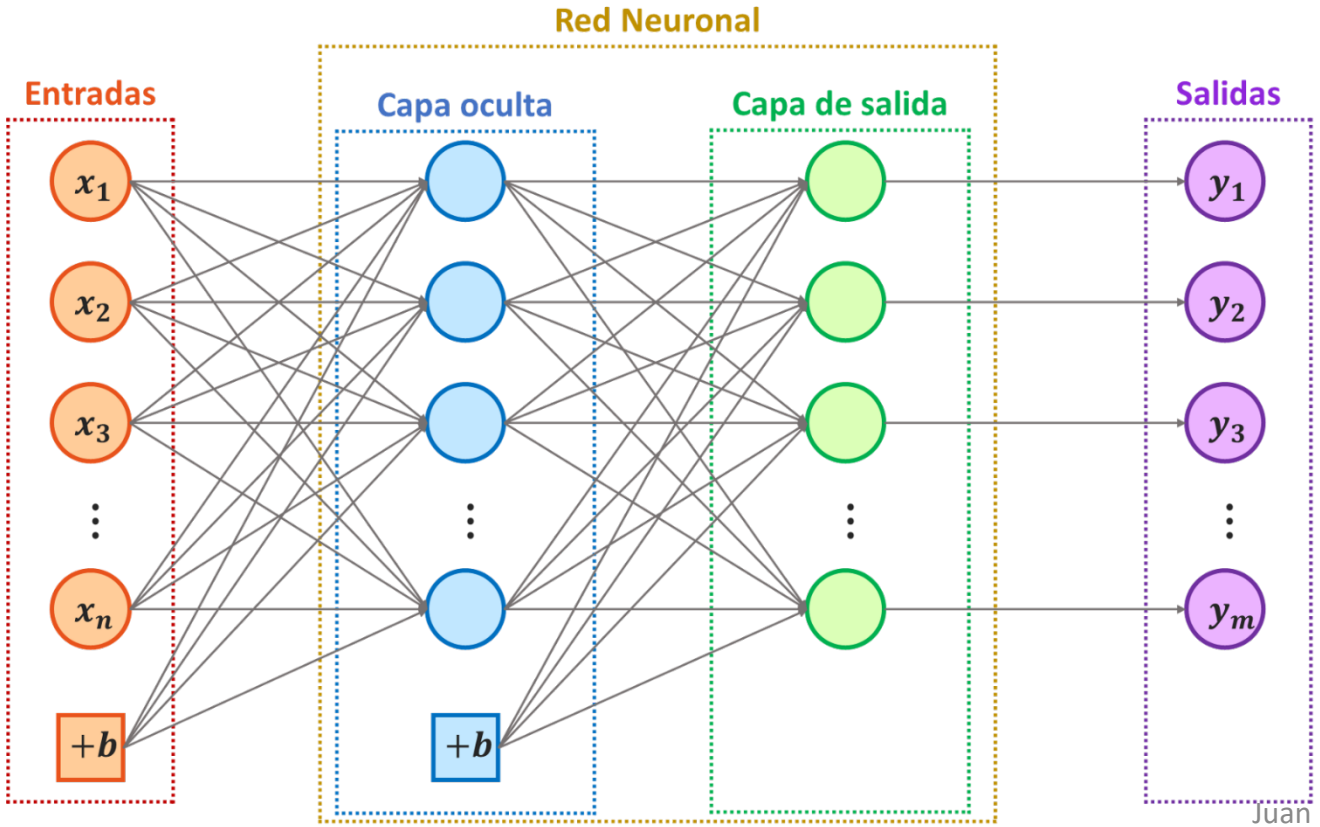
Máquinas de soporte vectorial (SVM)

Las máquinas de soporte vectorial buscan encontrar un hiperplano que separe de forma óptima a los puntos que componen diferentes categorías unos de los otros. Este tipo de algoritmo suele utilizarse para predecir a que categoría pertenecerá un nuevo punto del que no se tenía información con anterioridad.

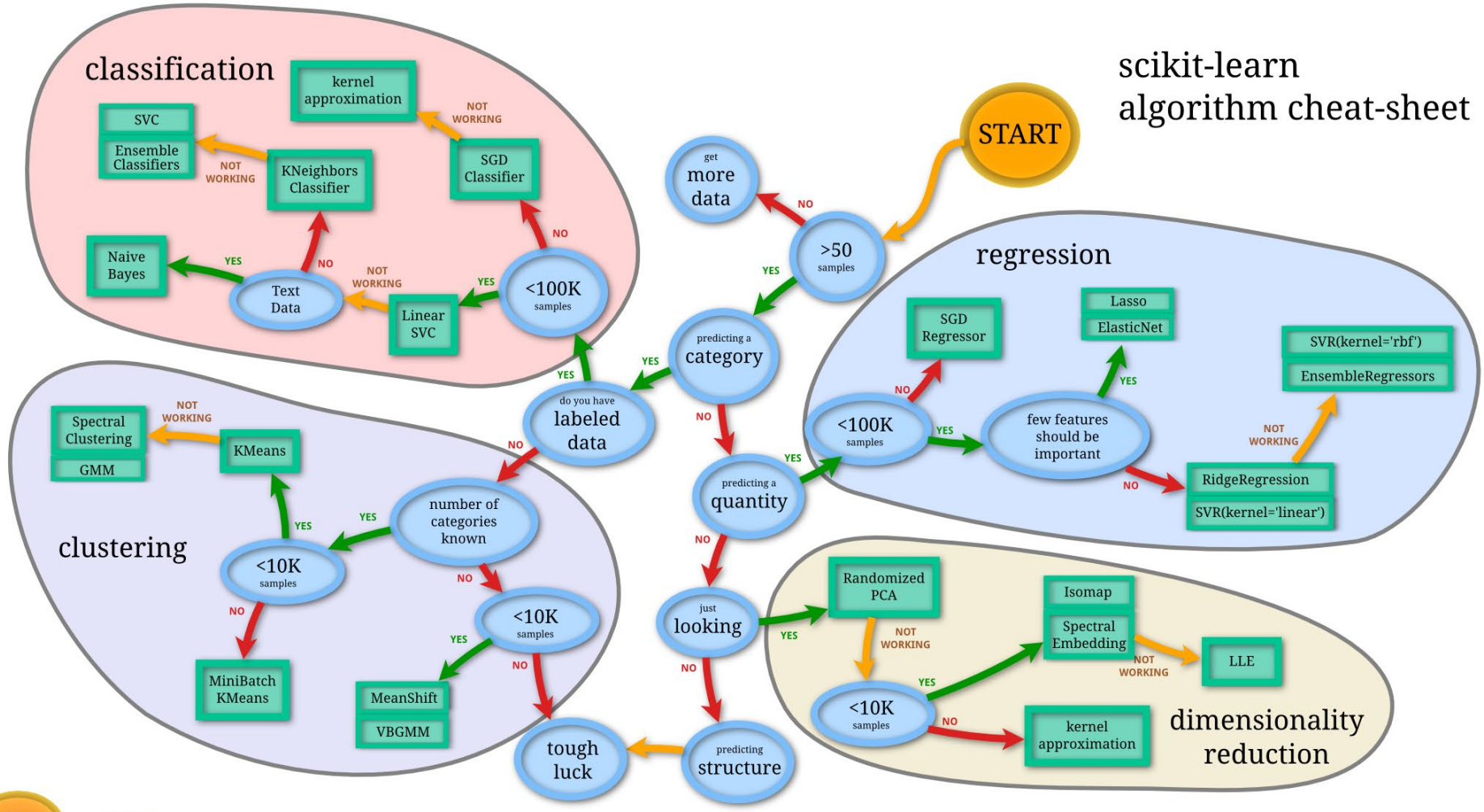


Redes neuronales

Las **Redes Neuronales Artificiales** son modelos que están inspirados en la biología, esto significa que están formados por elementos que se comportan de manera análoga a las neuronas del cerebro humano. Consiste en un conjunto de neuronas artificiales, conectadas entre sí las cuales son capaces de transmitir señales entre ellas. La información que entra en la red atraviesa las diferentes capas de neuronas, donde se somete a las entradas a diferentes operaciones matemáticas que secuencialemente producen modificaciones a estas entradas hasta obtener diversos valores de salida.



Como identificar el tipo de problema



Tipos de aprendizaje en ML:

- Aprendizaje supervisado
- Aprendizaje no supervisado
- Aprendizaje semisupervisado
- Aprendizaje activo
- Aprendizaje de transferencia
- Aprendizaje por refuerzo

Uno de los desafíos en el aprendizaje automático es obtener conjuntos de datos de muestra para la experimentación. En ML resulta útil contar con datos pata comprobar que algoritmo funciona correctamente. Existen numerosas bases de datos de todo tipo, destacamos aquí algunas de las más comunes:

- Bases de datos propias de Scikit-learn y Seaborn
- Conjunto de datos de Kaggle
- Depósito de ML UCI (Universidad de California, Irvine)

OTROS

- https://wiki.pathmind.com/open-datasets
- https://www.movebank.org/
- https://pfam.xfam.org/
- https://www.cs.waikato.ac.nz/~ml/weka/index.html
- https://en.wikipedia.org/wiki/List_of_datasets_for_machine-learning_research

Alguna librerías útiles para “data cleansing”:

- Webbrowser
- Request (solicitudes HTTP)
- Bs4 (Beautiful soup): páginas sin csv
- Bases de datos propias de Scikit-learn y Seaborn

Bases matemáticas

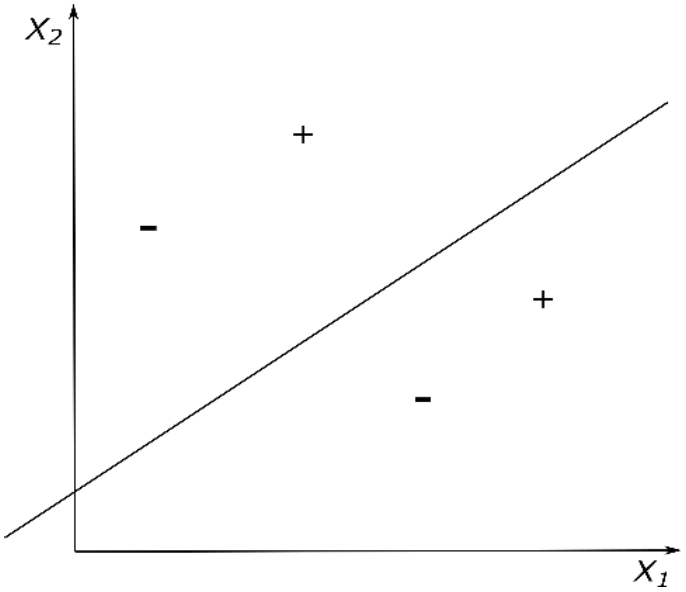
En los procesos de clasificación se asocian unas características (featured vector) con una etiqueta (label)

$$X^i = [1\ 0\ 0\ 1\ 0\ 0]$$

$$Y$$

$$\delta_n = \{(X^i, Y^i), i = 1,2 \dots, n\}$$

conjunto de los datos



$X \in \mathbb{R}^2$ e $Y = \{1, -1\}$

La idea general: clasificador h

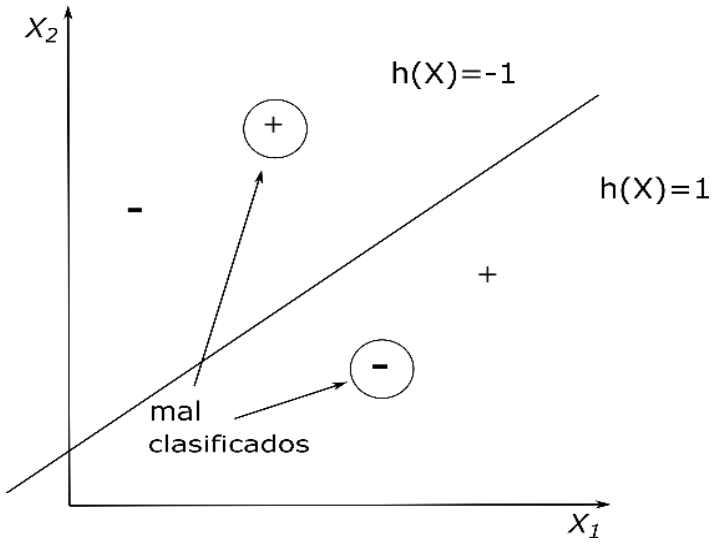
$$h: \chi \to \{1, -1\}$$

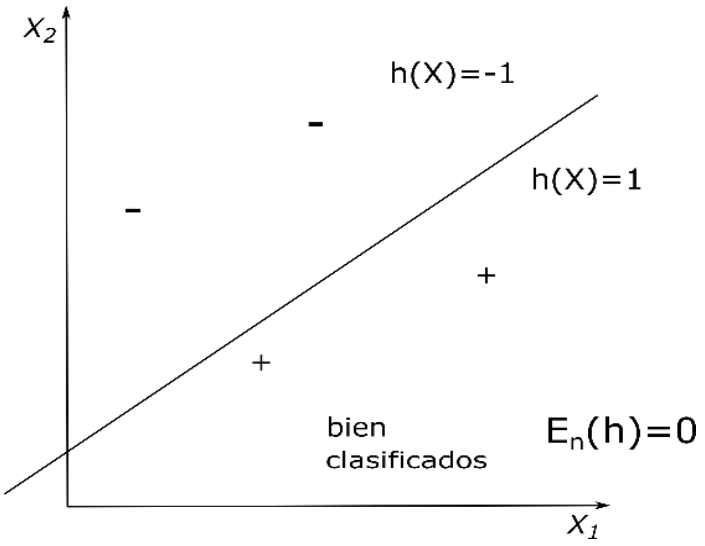
$$\chi = \mathbb{R}^2$$

Evaluación del clasificador (training set)

$$E_n = \frac{1}{n}\sum_{i=1}^{n} \left\| h(X^i) \neq Y^i \right\|$$

TRUE =1, FALSE=0.

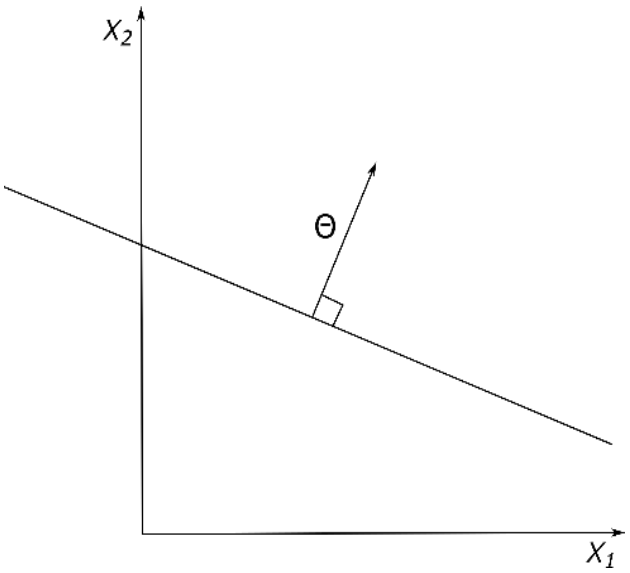




GENERALIZACIÓN: training set test set

Clasificador lineal

Frontera de decisión (decision boundary)



$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \qquad \theta = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}$$

Clasificador: obtener θ y θ_0 $\Rightarrow$
$\{h(x_i\theta, \theta_0) = sing(\theta x + \theta_0),\ \theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}\ \}$

Definición: un conjunto de entrenamiento $S_n = \{(X^i, Y^i), i = 1,2 \dots, n\}$ es linealmente separable si existe un vector de parámetro $\hat{\theta}$ y un offset θ_0 tal que $y^i(\hat{\theta}x^i + \theta_0) > 0 \quad \forall\ i = 1 \dots n$

Algoritmo Perceptrón (Frank Rosenblatt,1958).

Unidad básica de las ANN.

Su implementación estándar es:

1: **procedure** PERCEPTRON ($\{(x^i,y^i),\ i = 1,\ldots,\ n\}$, T)
2: $\theta = 0$ (vector), $\theta_0 = 0$ (escalar)
3: **for** $t = 1,\ldots,T$ **do**
4: **for** $i = 1,\ldots,\ n$ **do**
5: **if** $y^i(\theta\ x^i + \theta_0) \leq 0$ **then**
6: $\theta = \theta + y^i\ x^i$
7: $\theta_0 = \theta_0 + y^i$
8: **return** θ, θ_0

$$E_n = \frac{1}{n}\sum_{i=1}^{n}\left\|h(X^i) \neq Y^i\right\| \Rightarrow\ E_n(\theta,\theta_0) = \frac{1}{n}\sum_{i=1}^{n}\left\|y^i(\theta x^i + \theta_0) \leq 0\right\|$$

Ini. $\theta = 0$ => **if** $y^i(\theta\ x^i) \leq 0$ se cumple siempre => en el bucle actualizamos el vector $\theta = \theta + y^i\ x^i = y^i\ x^i$.

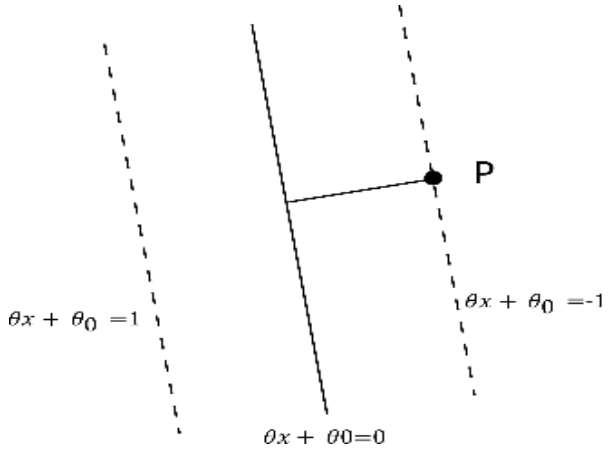
Volvemos al bucle y chequeamos de nuevo la condición : $y^i(y^i x^i)x^i = \left\|x^i\right\|^2 > 0$, con lo cual ahora funcionaría correctamente.

Resumiendo, los pasos a seguir hasta el momento son:

1. Definimos formalmente un conjunto de clasificadores
2. Introducimos parámetros que indexen los clasificadores del conjunto
3. Una vez caracterizado el conjunto de clasificadores necesitamos definir un algoritmo de aprendizaje que escoja el clasificador que mejor se adapte a nuestro problema.

Una vez conocido el clasificador lineal viene el problema de optimización:

- Large margin classifier
- Regularization
- Loss function



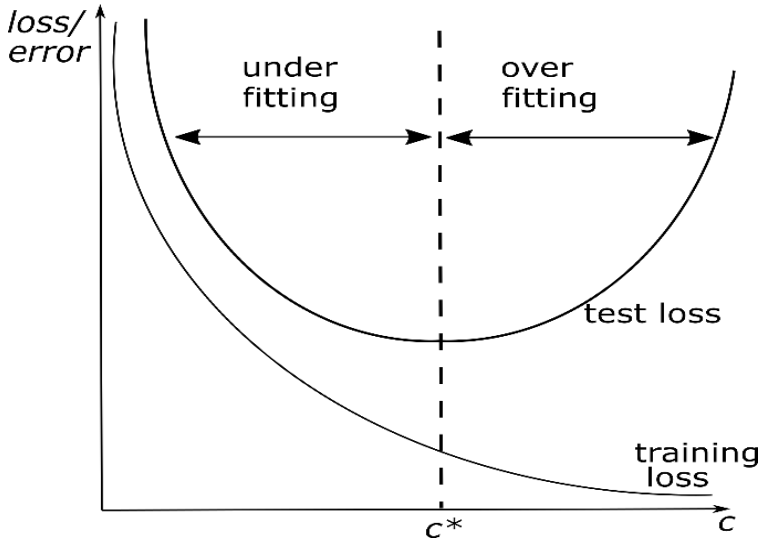
$$\gamma_i(\theta, \theta_0) = \frac{y^i(\theta x^i + \theta_0)}{\|\theta\|} = \frac{1}{\|\theta\|}$$

regulatization

$$J(\theta, \theta_0) = \frac{1}{n}\sum_{i=1}^{n} Loss_h\left(y^i(\theta x^i + \theta_0)\right) + \frac{\lambda}{2}\|\theta\|^2$$

λ parámetro de regularización

$$\frac{1}{\lambda}J(\theta,\theta_0) = \frac{1}{\lambda n}\sum_{i=1}^{n} Loss_h\left(y^i\left(\theta x^i + \theta_0\right)\right) + \frac{1}{2}\|\theta\|^2 \quad llamamos\ c = \frac{1}{\lambda n}$$

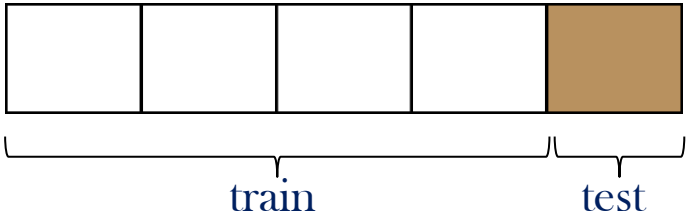


"c*" que minimiza los errores para ambos conjuntos

CROSS VALIDATION



Train set n=5 $c = \{c_1, \ldots . c_n\}$



Iter. 1 $acc = S(c_1)$



Iter. 2 $acc = S(c_2)$

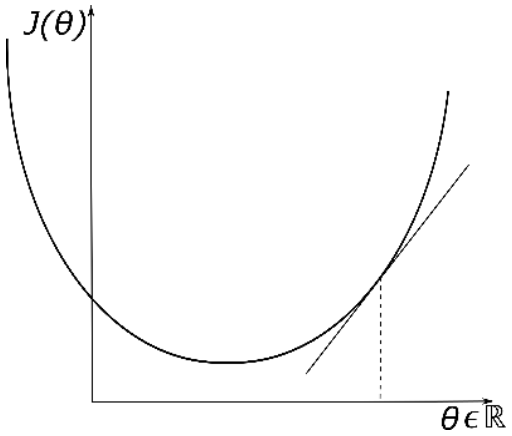


Iter. n $acc = S(c_n)$

$$c^* = argmax\, S(c)$$

Veamos ahora algunos algoritmos que nos permitan hallar dichas soluciones.

Método de descenso por gradiente



Partiendo de un punto aleatorio inicial, calculamos su pendiente, $\partial J(\theta)/\partial\theta$.

Para el segundo me desplazo en dirección opuesta al signo de la pendiente:

$$\theta \leftarrow \theta - \eta \frac{\partial J(\theta)}{\partial \theta}$$

η es la tasa de aprendizaje (step size o learning rate)

Generalizando la ecuación al espacio (j dimensiones):

$$\theta_j \leftarrow \theta_j - \eta \frac{\partial J(\theta)}{\partial \theta_j} \quad \Rightarrow \bar{\theta} \leftarrow \bar{\theta} - \eta \nabla J(\theta)$$

Descenso en gradiente estocástico (SGD)

Para de conjuntos de datos muy grandes el uso del algoritmo de descenso por gradiente resulta muy costos. Para solucionar este problema se introdujo el SGD

Elección al azar de un pequeño conjunto del conjunto de datos.

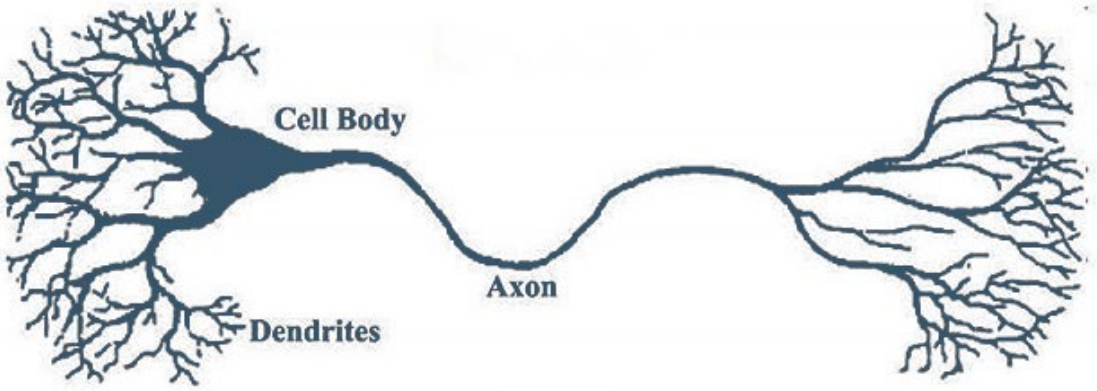
Es decir, partiendo de $$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} Loss_h\left(y^i \theta x^i\right) + \frac{\lambda}{2}\|\theta\|^2$$

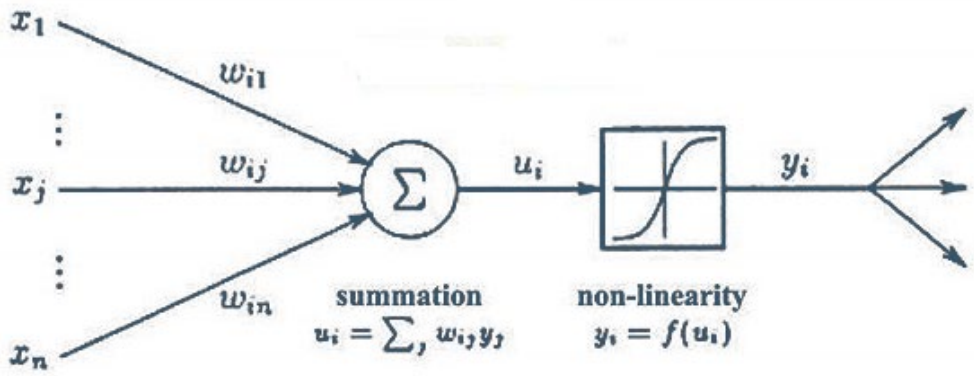
Seleccionamos $i \in \{1, \dots, n\}$ aleatorio y: $$\theta \leftarrow \theta - \eta_t \nabla_\theta \left\{\sum_{i=1}^{n} Loss_h\left(y^i \theta x^i\right) + \frac{\lambda}{2}\|\theta\|^2\right\}$$

Donde $$\eta_t \to 0 \quad \Rightarrow \quad \eta_t = \frac{1}{1+t}$$

Variante de este algoritmo: “Mini Batch SGD” (k=100)

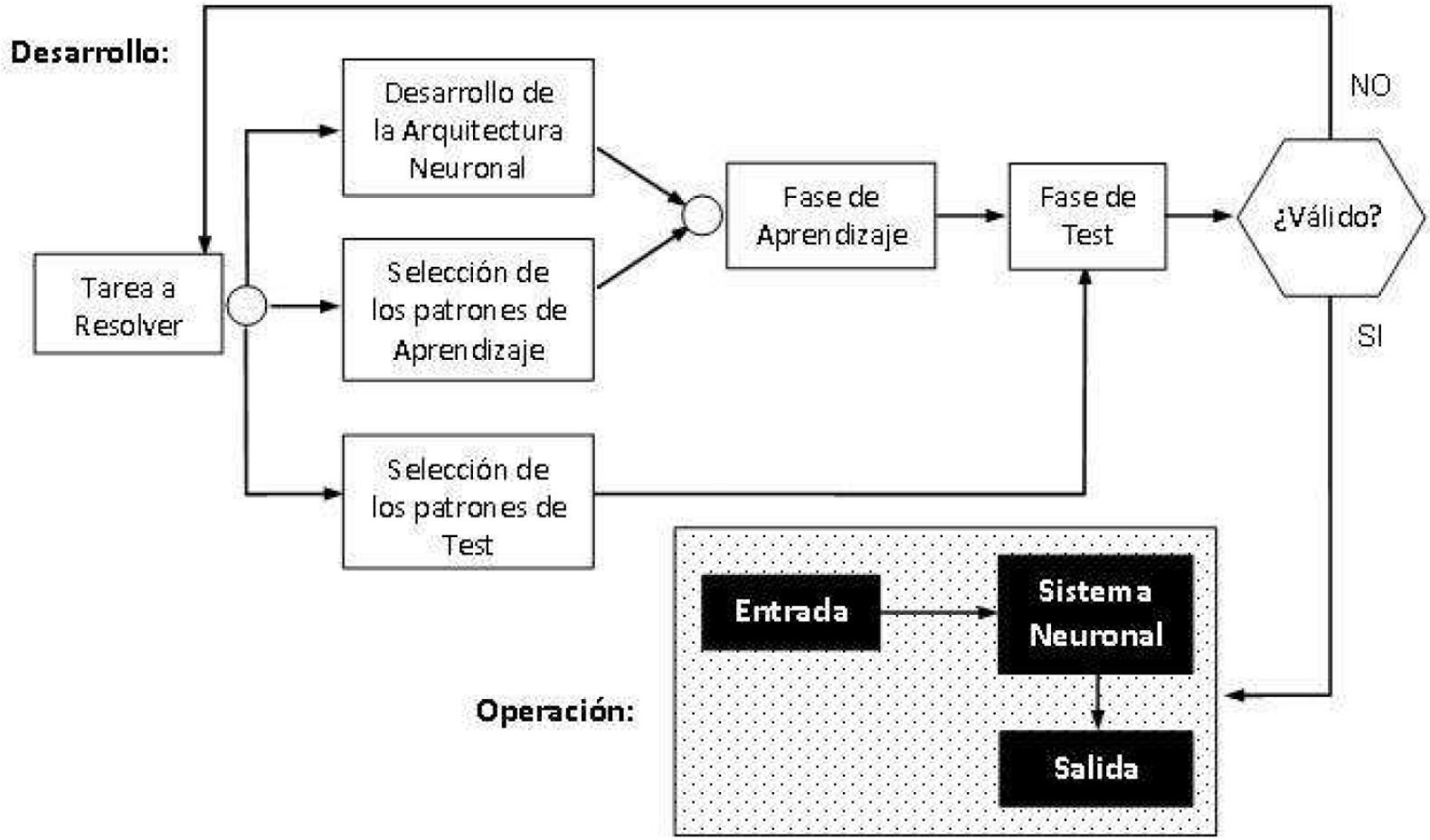
Redes de Neuronas Artificiales





Las RNA se han hecho muy populares debido a la facilidad en su uso e implementación y la habilidad para aproximar cualquier función matemática.

Las RNA son muy utiles para obtener resultados de datos complicados e imprecisos



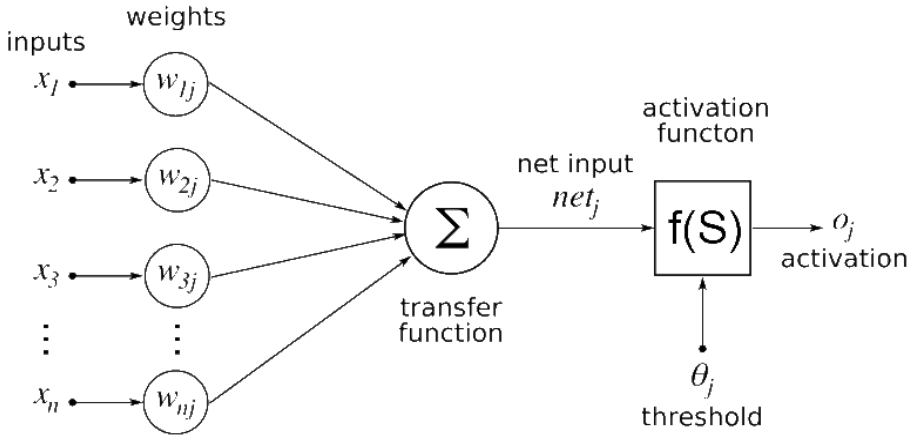
DEFINICION "Las redes neuronales son conjuntos de elementos de cálculo simples, usualmente adaptativos, interconectados masivamente en paralelo y con una organización jerárquica que le permite interactuar con algún sistema del mismo modo que lo hace el sistema nervioso biológico".

Gracias al masivo paralelismo de su estructura, gozan de una serie de ventajas:

- *Aprendizaje adaptativo.*
- *Autoorganización.*
- *Tolerancia a fallos.*
- *Capacidad de generalización.*
- *Operación en tiempo real.*

Contras: Al realizar un numero enorme de operaciones no es posible seguir paso a paso el razonamiento que les ha llevado a extraer sus conclusiones.

FUNDAMENTOS Y CONCEPTOS BÁSICOS



$$NET_i(t) = \sum_{j=1}^{N-1} [W_{ij} \cdot O_j \cdot (t-1)]$$

W_{ij} representa el peso de la conexión entre una neurona emisora j y una neurona receptora i.

La función de activación define el nuevo estado de activación

FUNCIÓN DE ACTIVACIÓN

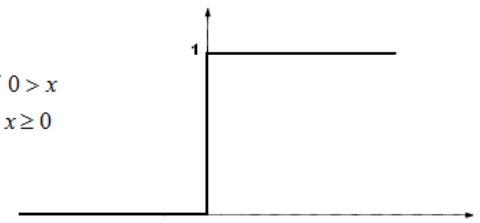
La función de activación de una neurona es la encargada de relacionar la información de entrada de la neurona con el siguiente estado de activación que tenga esa neurona. Existen dos modelos de función de activación:

- *Modelos acotados:*
- *Modelos No acotados*

El tipo de función de activación clasifica a las neuronas en lineales y no lineales.

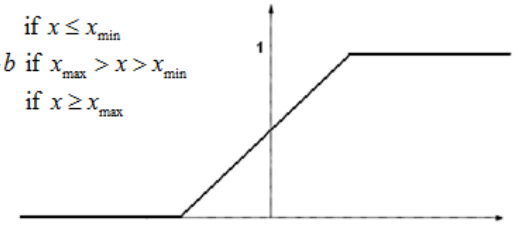
Unit step (threshold)

$$f(x)=\begin{cases}0 \text{ if } 0>x\\ 1 \text{ if } x\geq 0\end{cases}$$



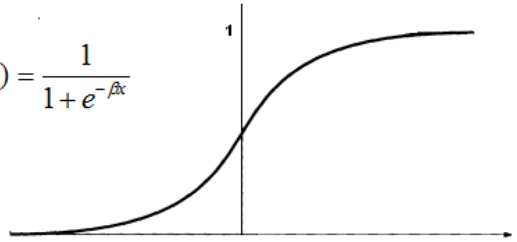
Piecewise Linear

$$f(x)=\begin{cases}0 & \text{if } x\leq x_{min}\\ mx+b & \text{if } x_{max}>x>x_{min}\\ 1 & \text{if } x\geq x_{max}\end{cases}$$



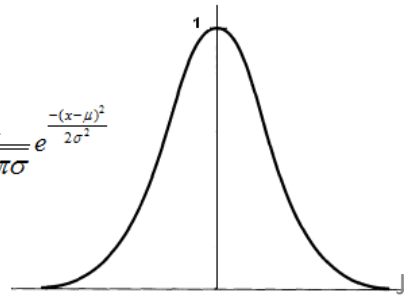
Sigmoid

$$f(x)=\frac{1}{1+e^{-\beta x}}$$



Gaussian

$$f(x)=\frac{1}{\sqrt{2\pi\sigma}}e^{\frac{-(x-\mu)^2}{2\sigma^2}}$$



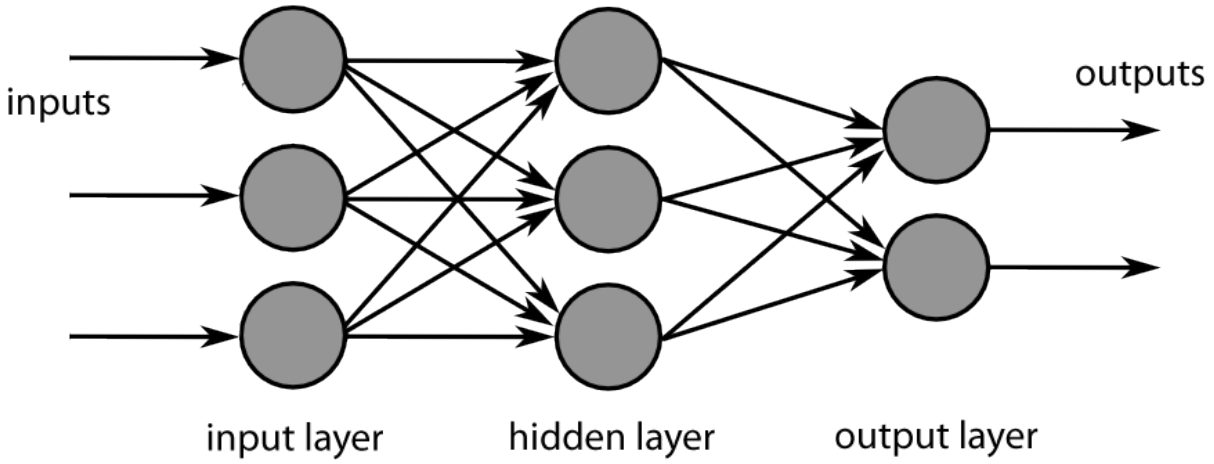
FLUJO DE DATOS EN LAS REDES DE NEURONAS ARTIFICIALES

Redes alimentadas hacia delante (Feed Forward)

La información se mueve en un único sentido, desde la entrada hacia la salida.

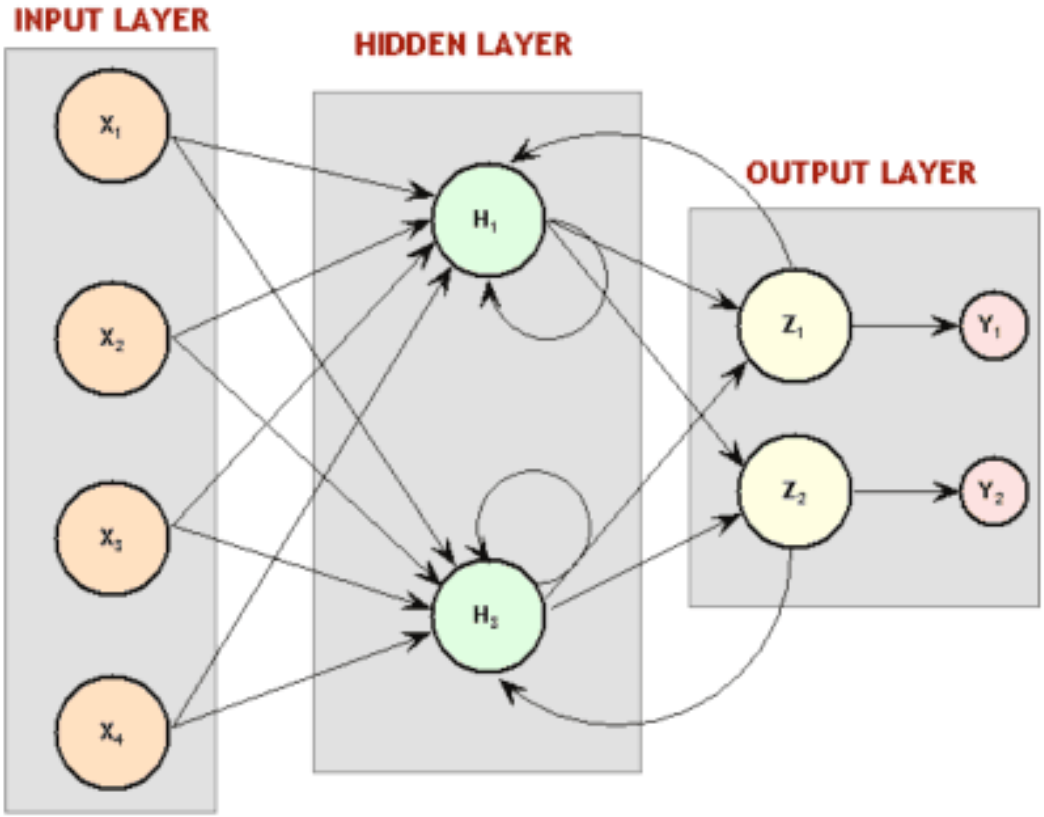
Organizadas en "capas". Cada capa agrupa a un conjunto de neuronas que reciben sinapsis de las neuronas de la capa anterior y emiten salidas hacia las neuronas de la capa siguiente.

Entre las neuronas de una misma capa no hay sinapsis



Redes con Retroalimentación total o parcial

Los elementos pueden enviar estímulos a neuronas de capas anteriores, de su propia capa o a ellos mismos: desaparece el concepto de agrupamiento en capas.



Las redes retroalimentadas emulan más fielmente la estructura del cerebro humano, en donde los fenómenos de retroalimentación son fundamentales

FASE DE CREACIÓN Y DESARROLLO

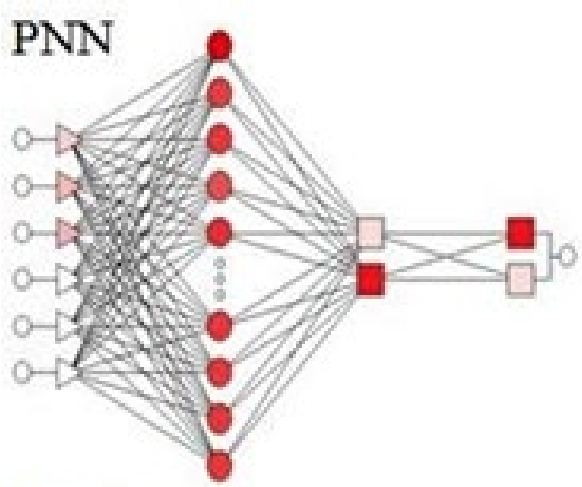
Esta etapa comprende todas las fases necesarias para el desarrollo de una RNA, y comprende las siguientes etapas:

- Diseño de la arquitectura.
- Entrenamiento de la red.
- Validación de la red.

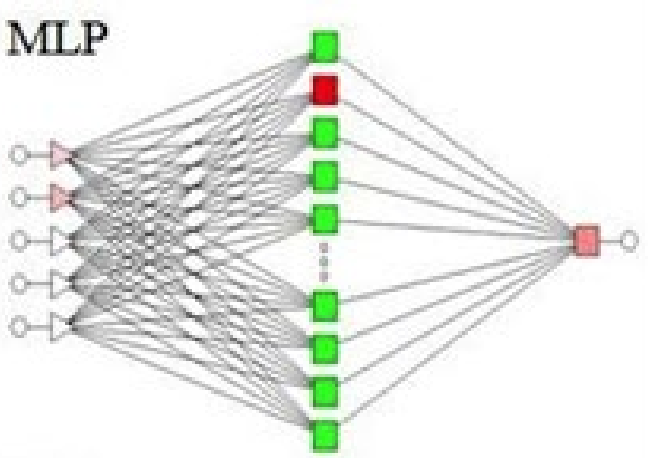
I. Diseño de la arquitectura

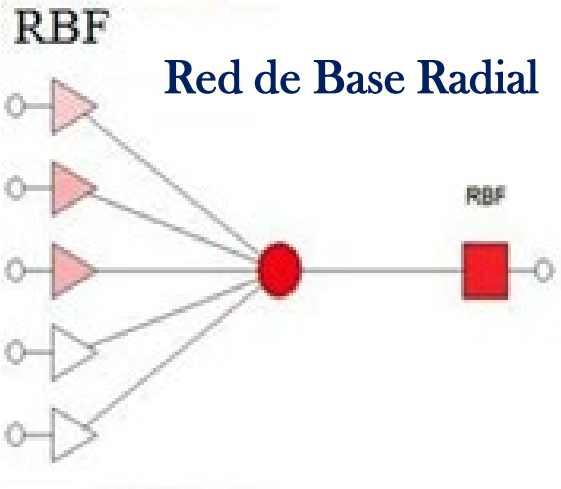
- Diseño de la topología
- Número de neuronas
- Disposición en capas
- Conectividad entre capas
- Funciones de activación

Red Neuronal Probabilística

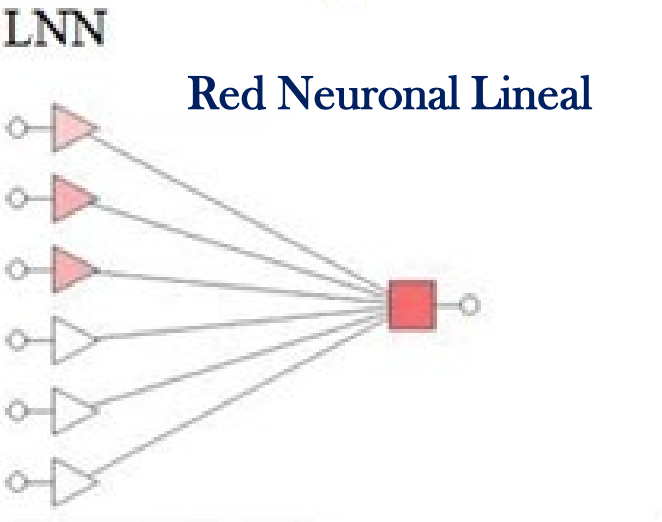


Perceptron Multicapa

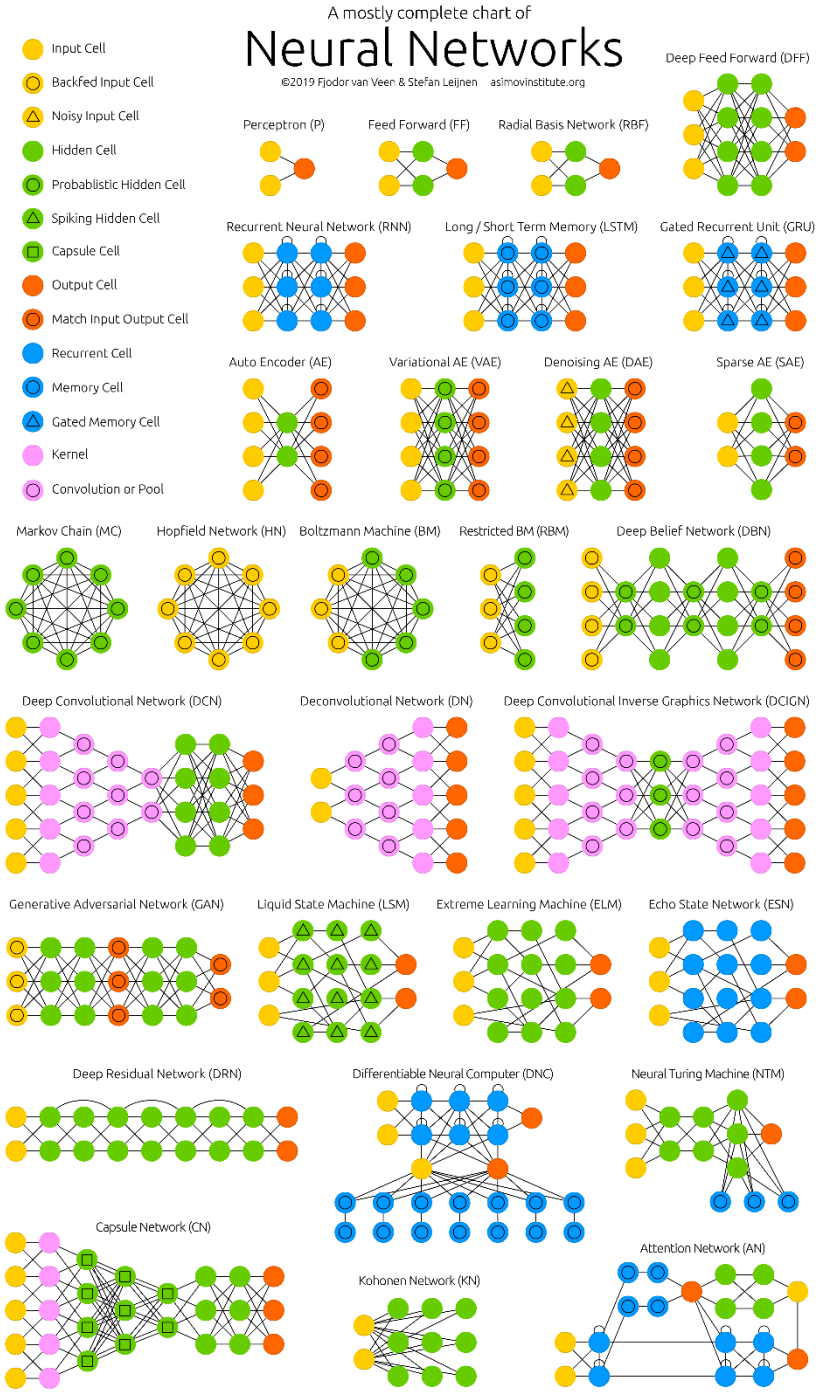




Red de Base Radial



Red Neuronal Lineal



Juan Ruso, Biofísica

II. Fase de Entrenamiento

Primero definir la condición de parada: se ha alcanzado una cuota de error lo suficientemente pequeña, se ha llegado a un número máximo de iteraciones definidas, se ha obtenido un error cero, se ha llegado a un punto de saturación donde el error no se reduce más.

Sobreentrenamiento: la red ha aprendido la información que se le ha suministrado en el proceso de aprendizaje a tal punto que ha modelado el ruido presente en la misma. Se puede producir por dos causas:

- Red neuronal Perceptron multicapa demasiado compleja. Esto significa que se tienen muchas neuronas en la capa o capas ocultas de la red neuronal
- El valor de los pesos es muy elevado. De manera heurística se ha detectado que cuando una red neuronal Perceptron multicapa esta sobreentrenada el valor de los pesos es grande

Solución

- Se eliminan neuronas y/o conexiones de tal manera que el error de la red no se vea afectado
- Se busca detener el entrenamiento en el momento en que el error de validación comienza a aumentar pues esto es un indicador de la perdida de generalización

Aprendizaje supervisado

Con esta técnica de aprendizaje el entrenamiento consiste en presentarle a la red repetitivamente patrones de estímulos de entrada pertenecientes a un juego de ensayo. El juego de ensayo esta formado por parejas "patrón de estímulos-respuesta correcta". En el juego de ensayo debe estar representada equilibradamente toda la información que la red necesite aprender.

Aprendizaje no supervisado

En este tipo de aprendizaje, no se especifica a la red cuál debe ser la respuesta correcta. Se necesita una cantidad mucho mayor de patrones de entrada durante el entrenamiento para que la red pueda ajustar correctamente sus pesos sinápticos.

Los procedimientos de aprendizaje son diferentes a los utilizados con el modelo de entrenamiento supervisado.

En este tipo de aprendizaje, lo que de hecho se está haciendo es exigirle a la red que capte por sí misma alguna de las características presentes en los datos de entrada.

Fase de validación o test

Para operar con una red entrenada, el proceso es el mismo que cuando se realizaba el entrenamiento. Por lo tanto, se le sigue suministrando información de entrada a la red, sólo que ahora no se realizara ningún ajuste en los pesos sinápticos.

Si los datos de un problema se diferencian demasiado de los del ensayo, la red tendrá dificultades para encontrar la respuesta correcta.

Los datos del ensayo deberán cubrir la totalidad de las características a las que la red debe de enfrentarse. Decidir cuantos datos se incluyen en el ensayo no es trivial.
A una red evaluadora es importante mostrarle tanto los patrones de entrada que llevan a tanto a evaluaciones positivas como negativas.
Se deben seleccionar aquellos que reflejen claramente los patrones a reconocer y las situaciones extremas de evaluación.
Lo ideal es preparar una colección amplia de hechos de entrenamiento procurando que todos los problemas queden bien representados.

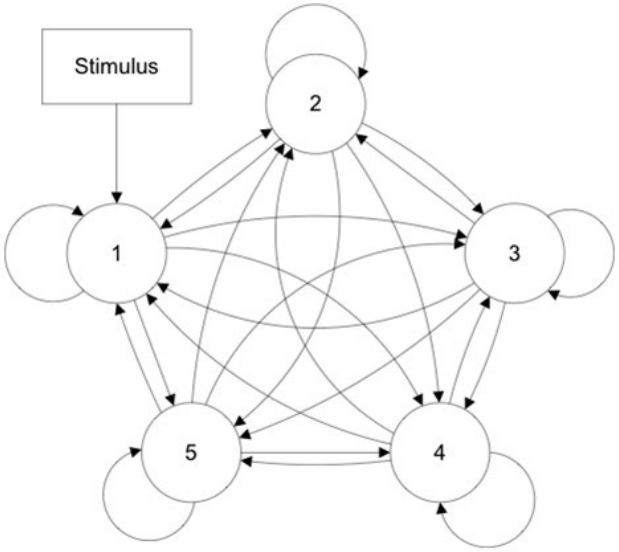
REDES RECURRENTES

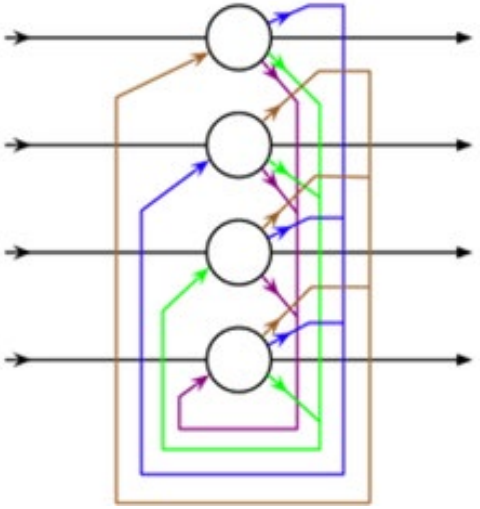
Cuando se trabaja con patrones dinámicos (t) las RNA feedforward se encuentran bastante limitadas.

La principal característica de las redes recurrentes es la de realimentar su salida a su entrada, evolucionando hasta un estado de equilibrio donde proporciona la salida final de la red. Esta característica las hace útiles cuando se quiere simular sistemas dinámicos; pero su entrenamiento es más lento complejo.

Pueden presentar otros problemas como comportamientos oscilatorios o caóticos.

El poco conocimiento es empírico y no existen estudios formales ni de la red recurrente más simple: una neurona aislada con una conexión así misma.





REPRESENTACION GRÁFICA DE LA EXACTITUD



SENSIBILIDAD: habilidad para detectar los verdaderos positivos.

- Todo verdadero positivo ha de poder ser detectado.

$$S = \frac{VP}{VP + FN}$$

ESPECIFICIDAD: capacidad de detectar aquello para lo que ha sido creado el clasificador.

- Todo falso negativo debería poderse detectar.

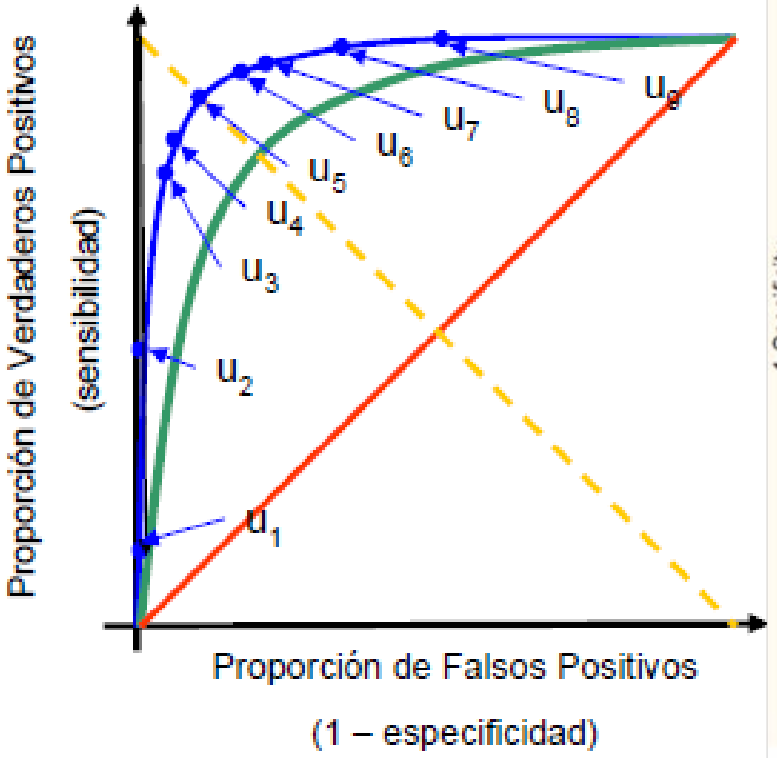
$$E = \frac{VN}{VN + FP}$$

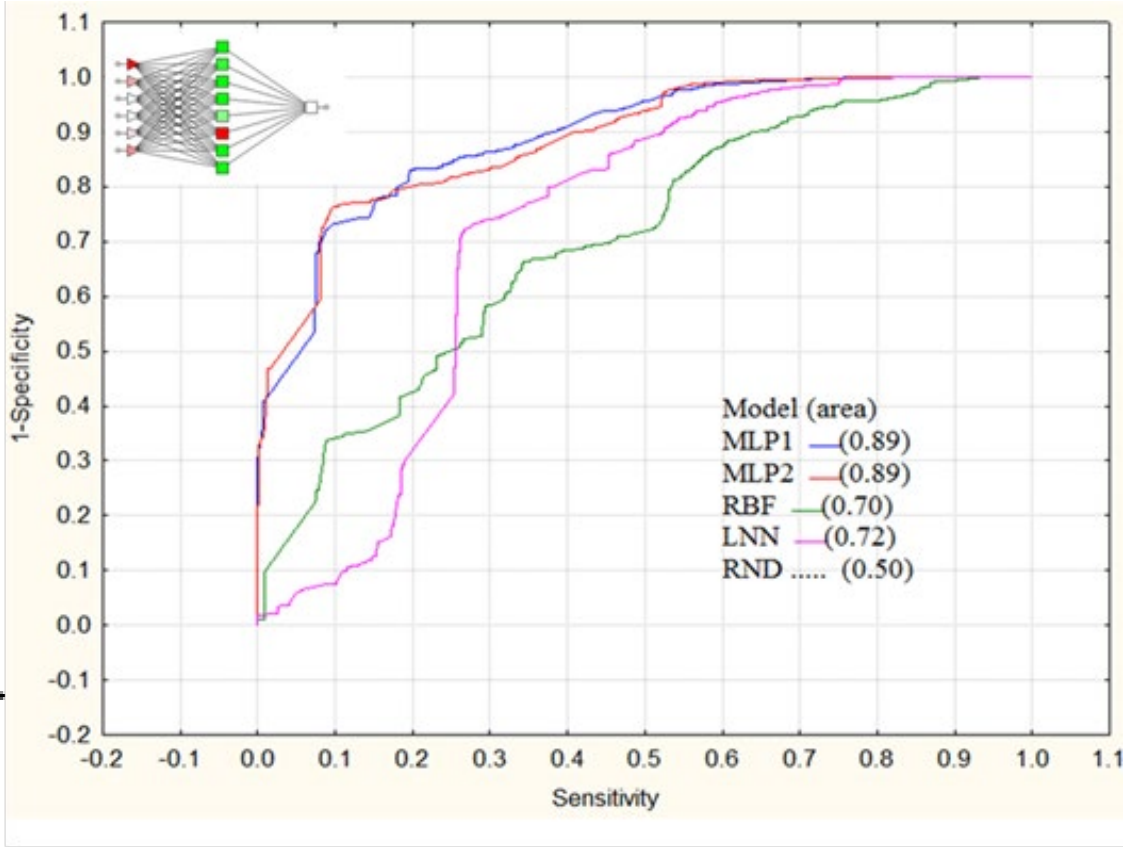
Idealmente S = 1.0 y E = 1.0, pero es muy difícil de obtener.

Hay un balance entre sensibilidad y especificidad: un aumento en una se traduce en una reducción de la otra.

Curva ROC (Característica Operativa del Receptor/Receiver Operating Characteristic)

Habrá una curva por cada clasificador. La eficiencia se mide por la superficie bajo la curva. Las curvas exteriores son mejores.

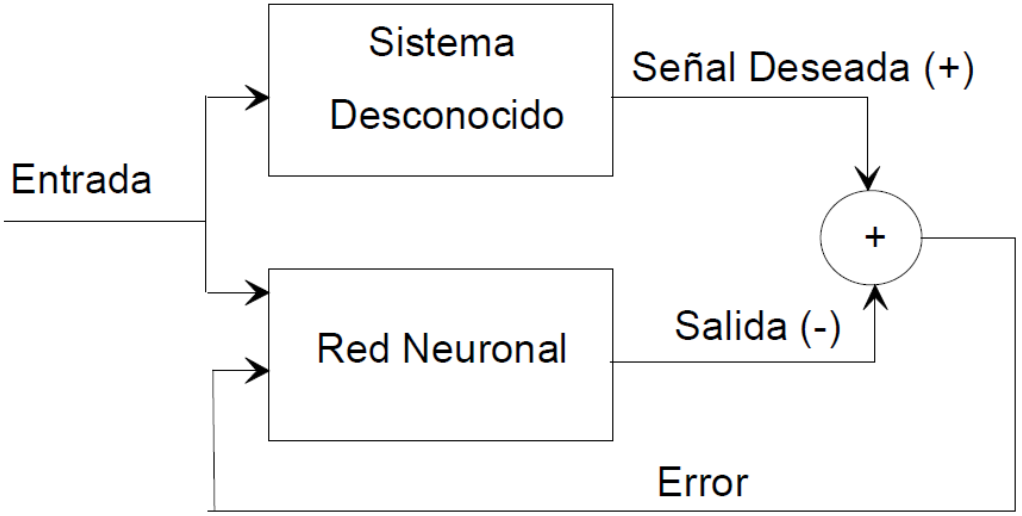




EJEMPLOS

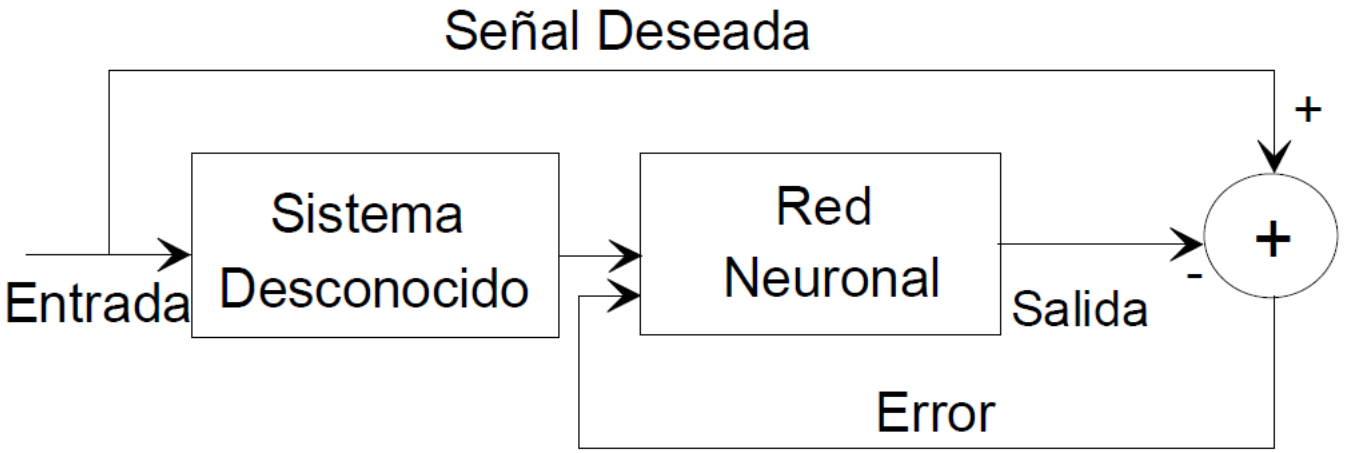
Las redes neuronales se pueden usar en una serie de estructuras según la aplicación a la que está destinado el sistema. Así pues, según la disposición de la red neuronal se tendrán las siguientes estructura

ESTRUCTURA DIRECTA



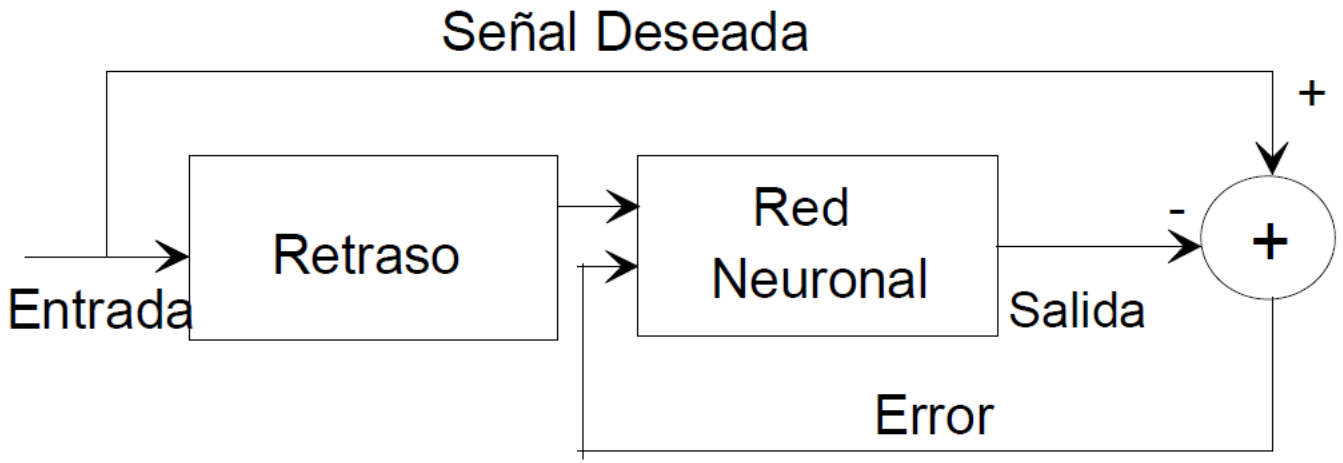
El sistema, en principio desconocido, y la red neuronal tienen las mismas entradas por lo que se conseguirá el mínimo error (objetivo de la red neuronal) cuando la salida de la red neuronal y la señal deseada sean iguales, es decir, cuando la función de transferencia de la red neuronal sea igual a la del sistema desconocido. Así pues esta estructura tiene como finalidad la modelización de funciones de transferencia de sistemas de los que, en principio, no conocemos nada pero tenemos la posibilidad de excitarlos con una determinada entrada y así conocer su salida

ESTRUCTURA INVERSA



El mínimo error en esta estructura se obtendrá cuando la salida de la red neuronal sea la entrada al sistema desconocido lo que conlleva que la función de transferencia de la red neuronal sea la inversa del sistema desconocido. Hay que destacar que el perfecto funcionamiento de esta estructura depende de la estabilidad de la inversa de la función de transferencia del sistema desconocido. Aplicaciones típicas de esta estructura son la ecualización de canales de comunicación y la resolución de problemas de deconvolución.

ESTRUCTURA CON RETARDO



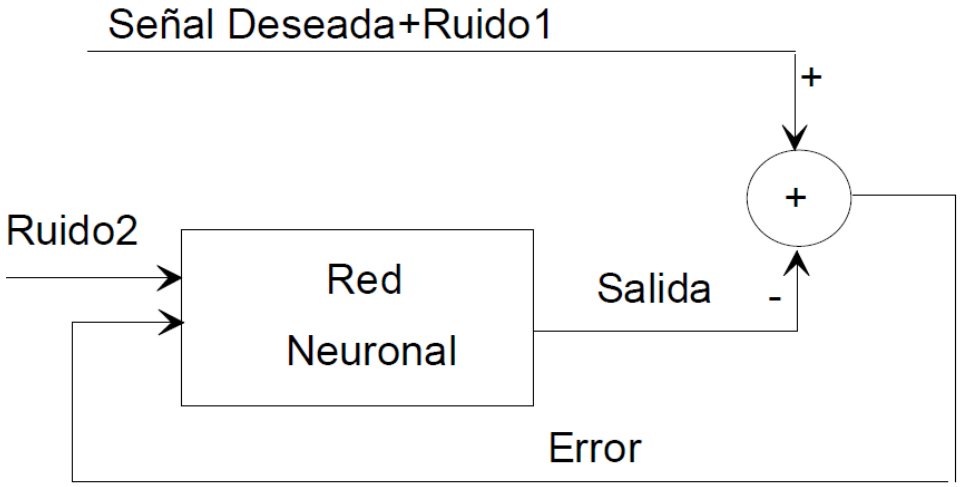
Esta estructura tiende a minimizar la diferencia entre la señal deseada (señal de entrada en el instante n) y la salida de la red neuronal que será un determinado valor obtenido con valores de la señal. Se intenta, pues, modelizar la señal actual a partir de los valores anteriores de ésta. Este sistema se puede usar, pues, en problemas de predicción (a partir de las muestras pasadas se puede estimar la siguiente) y de control (si se conoce la evolución del sistema se puede alterar los parámetros de dicho sistema para cambiar dicha evolución)

CANCELADOR DE RUIDO

Aquí se trata de minimizar la siguiente cantidad

$$e^2(n) = (señal\ deseada + ruido1 - salida)^2$$

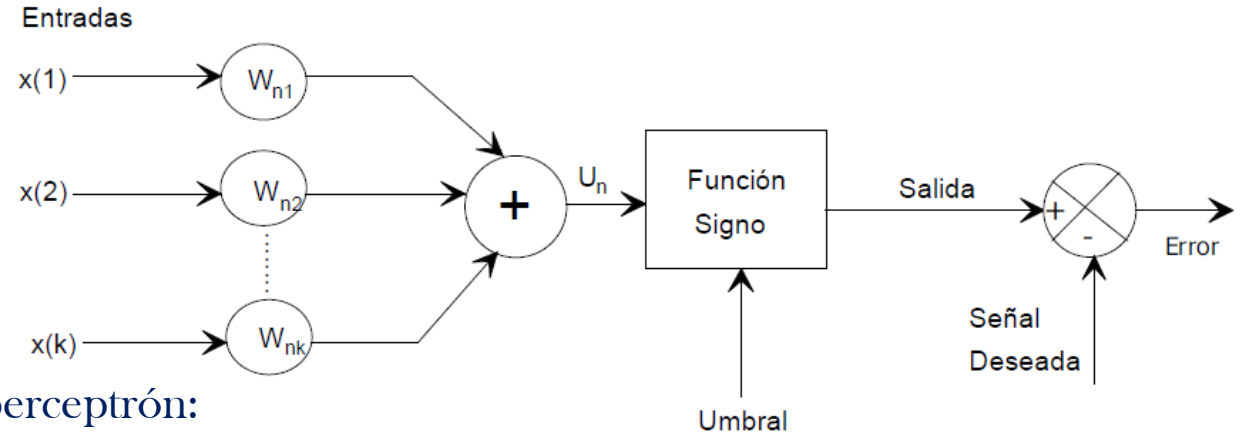
La condición de funcionamiento es la correlación entre el ruido que entra a la red neuronal y el que hay en la señal de referencia (señal a limpiar).



A nivel práctico este requerimiento es, la mayoría de las ocasiones relativamente sencillo de cumplir. Pensemos, por ejemplo, en la cancelación de ruido en un teléfono móvil. Aquí la señal de referencia sería nuestra voz contaminada con el ruido ambiental (señal de entrada al micrófono del teléfono) y la entrada a la red neuronal sería la señal que se tomaría en otro punto cercano al teléfono. Es necesario tener como precaución situar este último sensor de tal forma que nuestra voz (la señal de interés a recuperar) no aparezca en la entrada a la red neuronal.

De acuerdo con la expresión del error este será mínimo cuando la salida de la red neuronal sea igual al ruido que aparece en la señal de referencia ya que no existe ninguna relación entre la señal deseada y el ruido de la entrada a la red. En el momento de conseguir la minimización la salida de la red neuronal será el ruido1 y, por tanto, la señal de error será igual a la señal deseada, es decir obtendremos la señal limpia de ruido

EJEMPLOS
Perceptrón simple



Proceso de aprendizaje del perceptrón:

1. Inicialización de los pesos
2. Determinación de la salida: $y = \sum_{k=1}^{N} w_k^n \cdot x_k^n$

3. Comparación con el umbral: $u = y - umbral$

4. Aplicación de la función signo: $o = signo(u)$

5. Comparación con la señal deseada

6. Actualización de los coeficientes si existe error: $w_{k+1} = w_k + \eta(d - o)x_k$

Tomando una función signo (1 si o>0 y 0 si o<0), la frontera de decisión se toma para o=0. Tomemos un caso con dos entradas, x1 y x2. Si la salida, tras aplicar la función signo, vale cero significa que (w_0 es el umbral):

$$o = w_0 + w_1 x_1 + w_2 x_2 \qquad\qquad x_2 = -\frac{w_0}{w_2} - \frac{w_1}{w_2} x_1$$

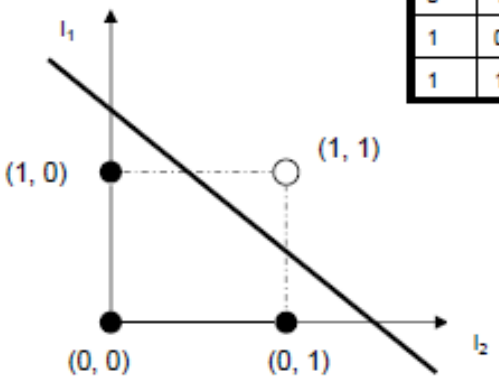
que es la ecuación de una recta (superficie de decisión en 2D).

De esta forma con un perceptrón podemos diseñar puertas AND y OR:

AND

w_1=1, w_2=1, θ=1.5

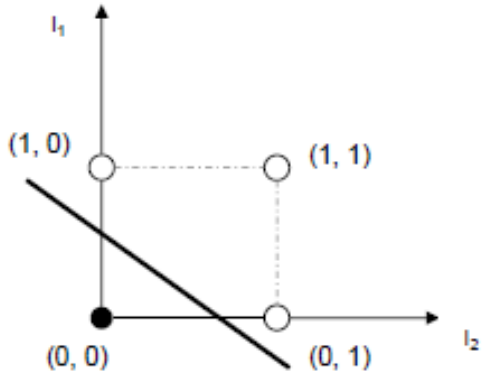
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1

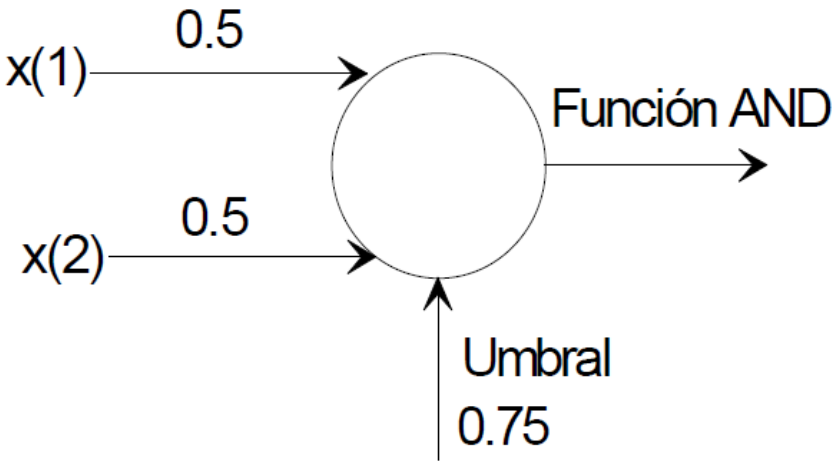


OR

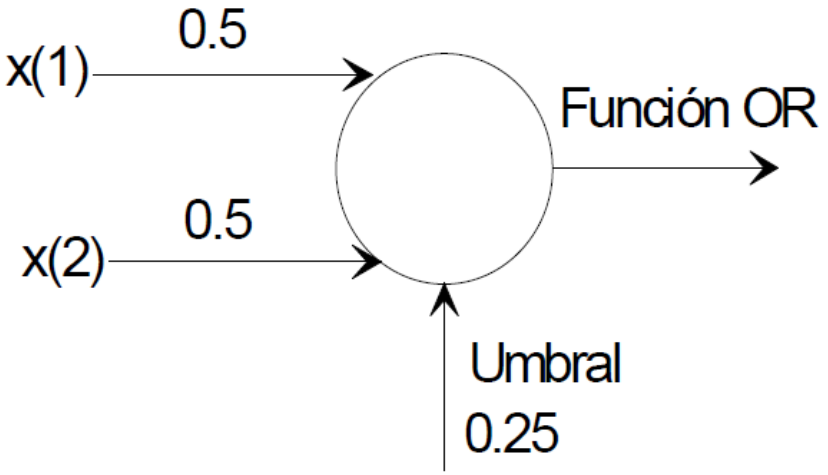
w_1=1, w_2=1, θ=0.5

OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1





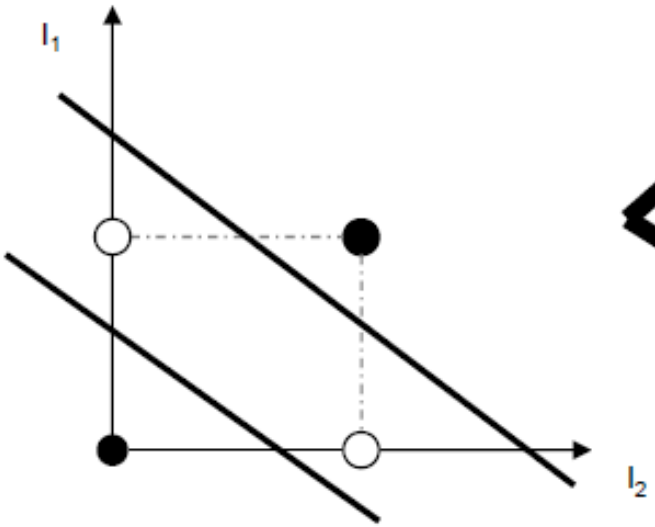
x(1)	x(2)	salida
0	0	0 (0<0.75)
0	1	0 (0.5<0.75)
1	0	0 (0.5<0.75)
1	1	1 (1>0.75)

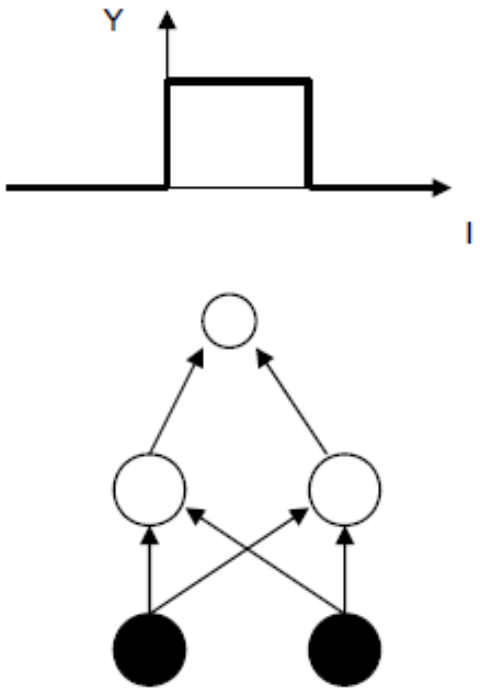


x(1)	x(2)	salida
0	0	0 (0<0.25)
0	1	1 (0.5>0.25)
1	0	1 (0.5>0.25)
1	1	1 (1>0.25)

Limitaciones del perceptrón: puerta XOR

XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



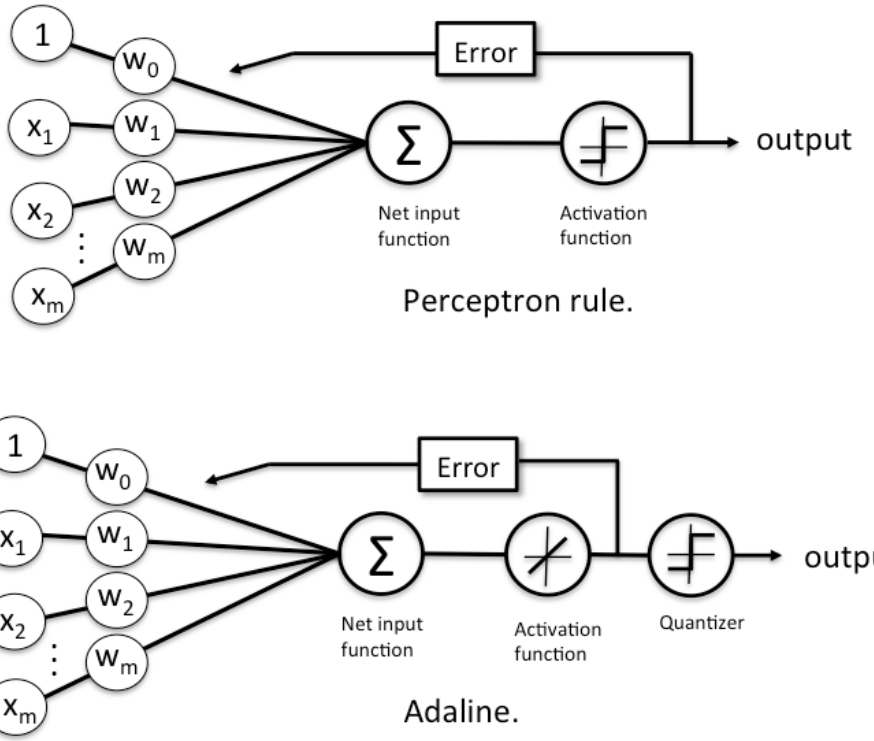


ADALINE: ADAptive LInear NEuron

La estructura Adaline es muy parecida al perceptron pero con el detalle de que la señal de error se calcula antes de aplicar la función signo. Este hecho permite enfocar el aprendizaje desde un punto de vista funcional: el sistema óptimo será aquél que asegure un error mínimo y, por tanto, se busca el mínimo de una función creciente del error.

Existen diferentes posibilidades a la hora de escoger esta función; la que se escogió a nivel histórico en primer lugar, y la que se usa en la gran mayoría de aplicaciones, es la función cuadrática, definida por:

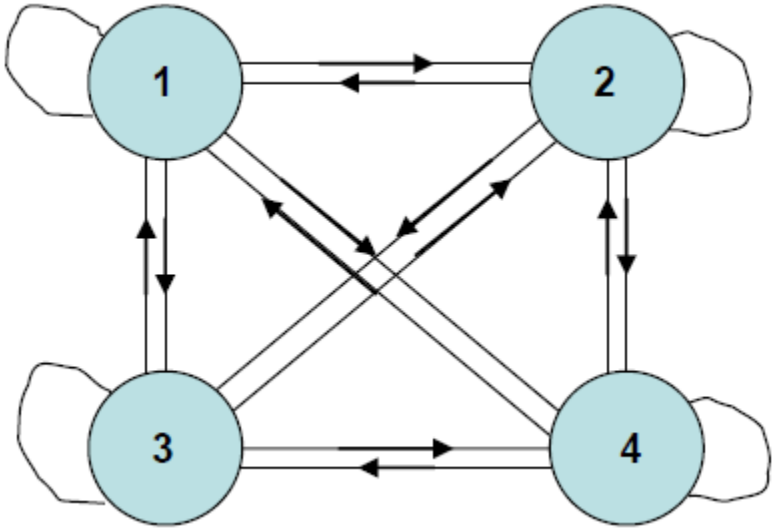
$$J(n) = error^2(n)$$



MEMORIA ASOCIATIVA

Se trata de un conjunto de neuronas interconectadas que evolucionan en el tiempo a partir de un estado inicial hasta estabilizarse en uno de los posibles estados finales

Ejemplo: 4 neuronas



Utilidad: para **recordar de forma estable**

Memoria asociativa

$s_1(t)$= Valor neurona 1 en el tiempo t
$s_2(t)$= Valor neurona 2 en el tiempo t
$s_3(t)$= Valor neurona 3 en el tiempo t
$s_4(t)$= Valor neurona 4 en el tiempo t



Evolución en el tiempo



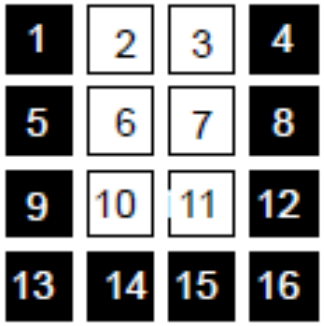
$s_1(t+1)= f(w_{1,1}s_1(t)+w_{1,2}s_2(t)+w_{1,3}s_3(t)+w_{1,4}s_4(t))$
$s_2(t+2)= f(w_{2,1}s_1(t)+w_{2,2}s_2(t)+w_{2,3}s_3(t)+w_{2,4}s_4(t))$
$s_3(t+3)= f(w_{3,1}s_1(t)+w_{3,2}s_2(t)+w_{3,3}s_3(t)+w_{3,4}s_4(t))$
$s_4(t+4)= f(w_{4,1}s_1(t)+w_{4,2}s_2(t)+w_{4,3}s_3(t)+w_{4,4}s_4(t))$

Patrón a recordar

$$w_{i,j} = \frac{1}{N} s_i s_j$$

Patrón para almacenar en memoria

$$s = (+1 \quad -1 \quad -1 \quad +1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \quad +1 \quad +1 \quad +1)$$



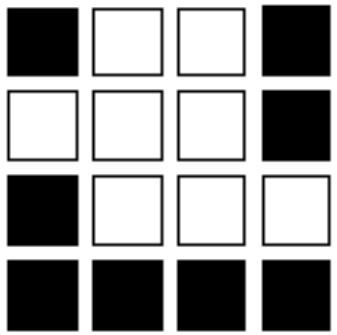
$$\begin{matrix} +1 & -1 & -1 & +1 \\ +1 & -1 & -1 & +1 \\ +1 & -1 & -1 & +1 \\ +1 & +1 & +1 & +1 \end{matrix}$$

$$\omega = s^T s = \frac{1}{16}$$

$$\begin{matrix}
1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 & 1 & 1 \\
-1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 \\
-1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 \\
1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 & 1 & 1 \\
1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 & 1 & 1 \\
-1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 \\
-1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 \\
1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 & 1 & 1 \\
1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 & 1 & 1 \\
-1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 \\
-1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 \\
1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 & 1 & 1 \\
1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 & 1 & 1 \\
1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 & 1 & 1 \\
1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 & 1 & 1 \\
1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 & 1 & 1
\end{matrix}$$

Patrón inicial defectuoso

$s(1) = (1 \quad -1 \quad -1 \quad 1 \quad -1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1 \quad -1 \quad 1 \quad 1 \quad 1 \quad 1)$

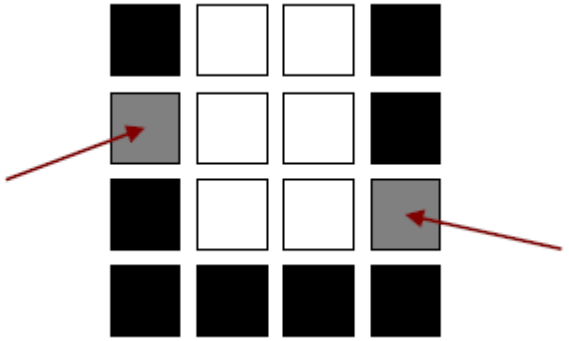


Evolución
(1 iteración)

$f(ws^T)$

Patrón final almacenado

$s(2) = (1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1)$



http://lcn.epfl.ch/tutorial/english/hopfield/html/index.html